UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA · EP INGENIERÍA DE SISTEMAS

INFORME DE PRUEBAS DE SEGURIDAD Y GESTIÓN DE CREDENCIALES EN PHARMABACKEND

Actividad Autónoma — Sesión 10 · Lenguaje de Programación II

Unidad: II. Frontend SPA empresarial seguro	**Ciclo / Semestre:** IV · 2026-2
Docente: Reyna Barreto Benjamin David	**Estudiante:** Gamarra
Repositorio: PharmaBackendv1	**Rama Git:** feature/seguridad-pruebas-gamarra
Entorno: Spring Boot 3.4.x, Java 21 LTS, Oracle 23ai Free	**Fecha de Entrega:** 18 de octubre de 2026

C1. Resumen Ejecutivo de Casos de Prueba Ejecutados

Se diseñó y ejecutó una batería integral de **28 casos de prueba automatizados** que abarcan la totalidad de vectores de seguridad: autenticación de usuarios, validación criptográfica de tokens JWT, control de acceso basado en roles y permisos (RBAC), operaciones administrativas sobre usuarios, preflight CORS y los tres nuevos endpoints requeridos en la Parte B. Todos los casos fueron verificados contra el backend en ejecución y la base de datos Oracle Database 23ai.

Grupo de Prueba	Casos Ejecutados	Pasan	Fallan	% Éxito
Autenticación (A-01 a A-05)	5	5	0	100%
Tokens JWT (T-01 a T-06)	6	6	0	100%
Permisos y Autorización RBAC (P-01 a P-05)	5	5	0	100%
Gestión de Usuarios (U-01 a U-03)	3	3	0	100%
CORS Preflight (C-01)	1	1	0	100%
Cambio de Contraseña B1 (B-01 a B-04)	4	4	0	100%
Asignación de Roles B2 (B-05 a B-07)	3	3	0	100%
Estado y Desactivación B3 (B-08)	1	1	0	100%
TOTAL CONSOLIDADO	**28**	**28**	**0**	**100%**

C2. Fichas de Hallazgos y Correcciones de Seguridad

A continuación se detallan tres hallazgos clave identificados y resueltos durante las fases de análisis, endurecimiento y pruebas del backend:

Ficha de Hallazgo: H-01 — Normalización de espacios en blanco en credenciales de login	Severidad: Media (Consistencia)
Descripción del Riesgo: Si un usuario remitía espacios antes o después del nombre de usuario (' vendedor '), la autenticación era rechazada con 401 debido a discrepancia de texto plano.	
Causa Probable: Clase: AuthServiceImpl | Método: login(). No se realizaba un saneamiento ni .trim() del campo username en el DTO recibido.	

Corrección Implementada: Se integró .trim() en el controlador y servicio, así como en UsuarioDetailsService.loadUserByUsername(), garantizando consistencia y previniendo falsos rechazos en interfaces web y móviles.

Ficha de Hallazgo: H-02 — Protección contra auto-desactivación y degradación del Administrador	Severidad: Alta (Denegación de Administración)
Descripción del Riesgo: Un usuario administrador con token válido podía invocar PUT /usuarios/{id}/roles retirándose el rol ADMIN o PATCH /usuarios/{id}/estado con false, dejando la plataforma acéfala y sin administradores habilitados.	
Causa Probable: Clase: UsuarioServiceImpl | Métodos: asignarRoles(), cambiarEstado(). Faltaba contrastar el identificador del usuario autenticado en el token con el registro objetivo.	
Corrección Implementada: Se extrae authentication.getName() y se valida: si usuario.getUsername().equalsIgnoreCase(adminUsername), se impide retirar el rol ROLE_ADMIN o setear estado=false, arrojando ReglaNegocioException (409 Conflict).	

Ficha de Hallazgo: H-03 — Expiración reactiva inmediata de tokens para usuarios deshabilitados	Severidad: Crítica (Acceso Post-Baja)
Descripción del Riesgo: En modelos JWT estándar sin estado, un token emitido permanece activo hasta exp. Si un administrador deshabilita a un usuario, este podría seguir operando hasta que venza el token.	
Causa Probable: Clase: JwtAuthenticationFilter | Método: doFilterInternal(). Depender exclusivamente de los claims del token en lugar de verificar el estado vivo en base de datos.	
Corrección Implementada: En JwtAuthenticationFilter se carga en cada petición UserDetails mediante userDetailsService.loadUserByUsername(username). Si usuario.isEnabled() es false, se bloquea la petición inmediatamente con 401 Unauthorized ('El usuario está deshabilitado').	

C3. Respuestas Argumentadas a las Preguntas de Análisis

1. ¿Por qué el login responde el mismo mensaje para un usuario inexistente, una contraseña incorrecta y una cuenta deshabilitada? ¿Qué información le daría a un atacante un mensaje distinto?

El endpoint de autenticación responde de forma estricta y deliberada el mismo código (HTTP 401 Unauthorized) y el mismo mensaje de error («Credenciales inválidas» o «Usuario o contraseña incorrectos») para evitar la vulnerabilidad de **enumeración de usuarios** (catalogada en OWASP ASVS y CWE-204). Si la API respondiera «El usuario no existe», un atacante podría probar listas masivas de correos o nombres (ataque de diccionario) para descubrir qué identidades son reales dentro de la botica y luego concentrar ataques de fuerza bruta o suplantación sobre ellas. Del mismo modo, si informara «La cuenta está deshabilitada», revelaría información sensible sobre el estatus contractual y el organigrama interno del personal. Al emitir una respuesta genérica e indistinguible, el atacante no obtiene ninguna confirmación probabilística que facilite su ataque.

2. Después de cambiar la contraseña (B1), el token anterior sigue siendo válido hasta que vence. ¿Por qué ocurre en una API sin estado? Propón una forma de invalidarlo.

Este fenómeno ocurre porque las arquitecturas basadas en JWT son por definición **sin estado (stateless)**: el servidor no almacena sesiones en memoria ni en base de datos para validar cada petición, sino que delega la autenticidad en la firma digital criptográfica del token y en la verificación de que la marca de tiempo actual sea menor a la fecha de expiración (claim exp). Por lo tanto, mientras la firma sea válida y no haya caducado, cualquier instancia del backend considerará legítimo el token emitido con la clave anterior. Para invalidarlo inmediatamente tras el cambio de contraseña se proponen dos mecanismos estándar de la industria: **a) Lista de revocación distribuida en Redis:** Al cambiar la contraseña, se publica el identificador único del token (claim jti) en una caché Redis con un tiempo de vida (TTL) igual al tiempo remanente del token; el JwtAuthenticationFilter consulta dicha lista en memoria ultra-rápida y rechaza los tokens revocados. **b) Campo de época de credenciales:** Guardar en la tabla usuarios el campo fecha_password_modificada. Al validar el token, si su fecha de emisión (claim iat) es anterior al timestamp del último cambio de contraseña, se rechaza de inmediato con 401.

3. Si el secreto del JWT se publicara en GitHub, ¿qué podría hacer un atacante? ¿Qué harías de inmediato?

Si la clave secreta HMAC (app.jwt.secret) se filtra públicamente, el esquema de seguridad queda totalmente vulnerado. Un atacante con el secreto puede generar (firmar) sus propios tokens JWT con cualquier contenido, adjudicándose la identidad de cualquier usuario (incluyendo el admin) y otorgándose permisos arbitrarios sin necesitar contraseñas ni registrarse en los logs del login. Esto permitiría acceso total a datos médicos y contables, alteración de inventarios y anulación de ventas. **Acciones inmediatas obligatorias:** 1) **Rotación urgente de la clave secreta:** Actualizar de inmediato el secreto en los servidores de despliegue mediante variables de entorno inyectadas (Key Vault / AWS Secrets Manager / variables del sistema), lo que invalida en el acto todos los tokens falsificados o legítimos emitidos previamente. 2) **Depuración y saneamiento del repositorio Git:** Emplear herramientas especializadas como git-filter-repo o BFG Repo-Cleaner para reescribir la historia de commits y eliminar la credencial expuesta de forma permanente, seguido de un *force-push* a las ramas remotas. 3) **Auditoría forense:** Inspeccionar los registros de acceso del servidor web y base de datos para identificar si hubo accesos anómalos previos a la rotación.

4. ¿Qué ventaja tiene proteger los reportes con @PreAuthorize en el controlador frente a una regla por URL en SecurityConfig? ¿Y qué desventaja?

Ventajas de @PreAuthorize: Proporciona seguridad contextual y granular colocada directamente junto a la firma del método en el controlador. Permite utilizar el lenguaje de expresiones de Spring (SpEL) para evaluar condiciones dinámicas complejas basadas en parámetros de entrada y en el objeto Authentication (ej. verificar si el usuario tiene permiso sobre su propia sucursal o si la fecha solicitada está autorizada). Además, hace que la seguridad sea resistente a refactorizaciones de rutas o prefijos de URL en la anotación @RequestMapping.
Desventajas de @PreAuthorize: Produce **dispersión de la política de seguridad**: las reglas quedan distribuidas entre múltiples clases controladoras en lugar de encontrarse centralizadas en un único archivo de auditoría como SecurityConfig, lo que dificulta obtener una vista panorámica de la superficie expuesta de la API. Asimismo, existe el riesgo de que un programador cree un nuevo método en el controlador y olvide anotarlo; si la regla global por defecto es permisiva, el endpoint quedará involuntariamente expuesto.

C4. Documentación Técnica de los Nuevos Endpoints (Parte B)

Se implementaron y probaron con éxito los 3 endpoints de gestión requeridos, respetando los contratos de datos y la arquitectura multicapa:

Endpoint y Método	Rol / Autorización	Entrada (Body / Path)	Respuesta y Códigos HTTP
PUT /api/v1/auth/password	Cualquier usuario autenticado	JSON: { passwordActual, passwordNueva }	**204 No Content** si es exitoso. 409 si clave actual no coincide o si nueva es igual. 400 si clave nueva no cumple regex.
PUT /api/v1/usuarios/{id}/roles	Solo ADMIN (USUARIOS_ADMINISTRAR)	Path: id (usuario) JSON: { rolIds: [2, 3] }	**200 OK** con UsuarioResponseDTO. 400 si rolIds está vacío. 404 si usuario o roles no existen. 409 si admin intenta quitarse el rol ADMIN.
PATCH /api/v1/usuarios/{id}/estado	Solo ADMIN (USUARIOS_ADMINISTRAR)	Path: id (usuario) JSON: { estado: false }	**200 OK** con UsuarioResponseDTO. 404 si usuario no existe. 409 si admin intenta desactivarse a sí mismo. Token previo deja de funcionar (401).

Conclusión

Con la culminación de esta actividad, PharmaBackend cuenta con una suite completa de seguridad verificada de 28 casos, gestión robusta de credenciales y roles con BCrypt, y protección ante vectores de auto-bloqueo y denegación de servicio administrativo. El backend queda 100% listo para integrarse en la Sesión 11 con el cliente frontend Single Page Application (SPA).